TED UNIVERSITY

Faculty of Engineering
Department of Computer Engineering
Project Name: F&D Autonomous Driving

Low-Level Design Report

Team Members:

Can Soysal
Çağrı Şanlı
Mustafa Cem Hür
Taha Ugan
Taylan Kayalı

Project Supervisor:

Tolga Kurtuluş Çapın

Project Jury:

Deniz Cantürk
Hakkı Gökhan İlk

Project Website:

[Autonomous Driving Algorithm - Final Project]

1. Introduction..... 3
1.1 Object Design Trade-offs..... 3
Reuse vs. Specificity..... 3
Performance vs. Cost..... 3
Energy Efficiency vs. Real-Time Responsiveness..... 3
1.2 Interface Documentation Guidelines..... 3
1.3 Engineering Standards..... 4
1.4 Definitions, Acronyms, and Abbreviations..... 5
2. Packages..... 5
3. Class Interfaces..... 7
3.1 Perception Subsystem (fd_perception)..... 7
3.1.1 VisionProcessor..... 7
3.1.2 ObstacleClassifier..... 7
3.1.3 TrafficSignRecognizer..... 8
3.1.4 DepthEstimator..... 8
3.1.5 LocalizationEstimator..... 9
3.2 Planning Subsystem (fd_planning)..... 9
3.2.1 RoadGraph..... 9
3.2.2 GlobalPlanner..... 10
3.2.3 LocalPlanner..... 11
3.2.4 BehaviorManager..... 12
3.3 Control Subsystem (fd_control)..... 13
3.3.1 TrajectoryTracker..... 13
3.3.2 EmergencyStop..... 13
3.3.3 ParkingController..... 14
3.4 Simulation Interface (fd_simulation)..... 14
3.4.1 GazeboSensorBridge..... 14
3.4.2 GazeboActuatorBridge..... 15
3.4.3 SimulationController..... 15
3.5 Design Patterns Summary..... 16
4. Glossary..... 17
5. References..... 19

1. Introduction

F&D Autonomous Driving System is a virtual autonomous driving simulation system that is designed to act as a platform for self-driving algorithms to safely test their logic to address any errors or miss' before they are tested on the actual hardware. F&D uses road network perception and path planning to simulate a vehicle that is able to converse elaborate scenarios without any human involvement.

1.1 Object Design Trade-offs

Reuse vs. Specificity

We use already existing platforms and software such as Gazebo and ROS2 to construct and connect our environment. This allows us to concentrate our efforts on optimizing our models rather than ground-up system construction which would otherwise make the system too time consuming and complex to make.

Performance vs. Cost

The system uses the GPU for 3D environment rendering and simulation while also accelerating neural network inference in the perception layer. Modern object detection models like YOLO can run much faster on GPUs than CPUs but it becomes costlier as the performance requirements increase. Some of the computational demands like vision processing is relegated to CPU using ROS2 which allows for components to run as separate processes therefore distributing the computational load across cores.

Energy Efficiency vs. Real-Time Responsiveness

The system's primary focus is its energy-aware path planning. Computing energy-optimal paths is significantly heavier and slower than computing shortest-distance paths. We overcome this issue by separating the planning layers into a global and local planning. This separation allows us to find the most optimal route using the global planner while the local planner generates detailed trajectories for a smaller time window which allows for faster response times.

1.2 Interface Documentation Guidelines

- All class interfaces in this document follow a uniform specification format to ensure consistency and traceability across subsystems.

- **Method Tables.** Each public method is documented in a three-column table: Method Signature, Pre-condition, and Post-condition / Returns. The Method Signature column gives the full Python-style signature including parameter types and return type. The Pre-condition column states what must be true before the method is called. The Post-condition / Returns column states what the method guarantees on success, including return types, published topics, and any exceptions raised (e.g. PlanningTimeoutError, SensorFusionError, InsufficientDataError).

- **Private vs. Public Methods.** Methods prefixed with an underscore (e.g. _preprocess, _aStar) are internal helpers and are included in the tables only when they carry a non-functional requirement (such as a timing or accuracy constraint) or are referenced by another subsystem's pre-condition.

1.3 Engineering Standards

To ensure maintainability, readability, and consistency across the F&D codebase, we follow coding standards based on the technologies we use.

By dividing the system into layers and different modules, needs and demands are effectively distributed and parts and components can be developed and tested individually which simplifies the process and allows for further scalability

Python (ROS2 Nodes and ML Pipeline):

- Keep lines short for readability and comply with the python syntax.
- Method and variable names use snake_case throughout (e.g., detect_lanes, block_edge).
- Use Python 3.10+ type hints for all method signatures.
- Organize imports in groups: standard library first, then third-party such as numpy, cv2; then local packages.
- Write docstrings for all public methods describing purpose, pre-conditions, and return values.

ROS2 (Communication and Package Structure):

- Topic and service names use snake_case and are prefixed by the subsystem (e.g., /perception/obstacles).
- All custom message definitions are centralized in fd_msgs to avoid circular dependencies.
- Each package manages only one subsystem and nothing more.

C++ / Gazebo Plugins:

- Class names use PascalCase while methods and variables use snake_case.
- All Gazebo-specific logic stays inside fd_simulation so the rest of the system remains simulator-agnostic.

For general principles, Modularity, Error Handling (using typed exceptions like SensorFusionError or PlanningTimeoutError instead of unexplainable failures), and Documentation are rules we follow throughout F&D development.

1.4 Definitions, Acronyms, and Abbreviations

API - Application Programming Interface

CPU - Central Processing Unit

FPS - Frames Per Second

GPU - Graphics Processing Unit

MPC - Model Predictive Control

PID - Proportional-Integral-Derivative (Control)

RGB - Red-Green-Blue (Color Model)

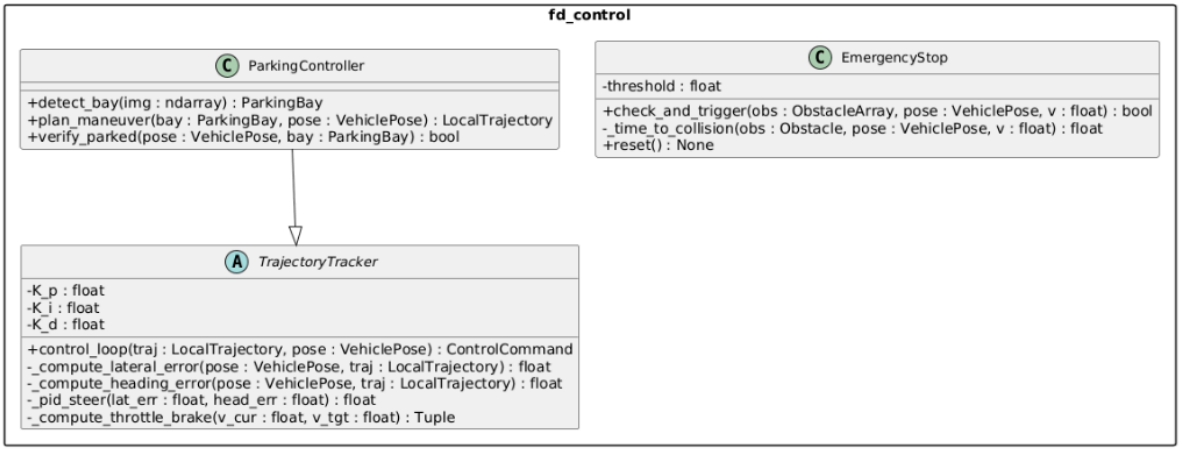
ROS2 - Robot Operating System 2

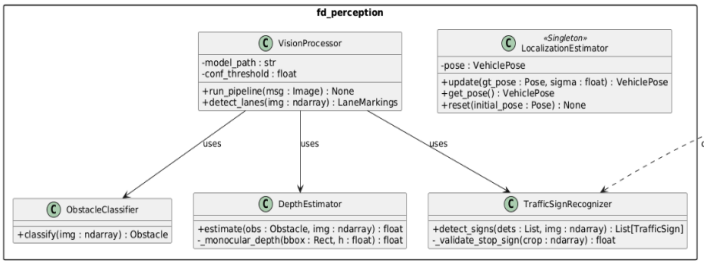
YOLO - You Only Look Once (Object Detection Algorithm)

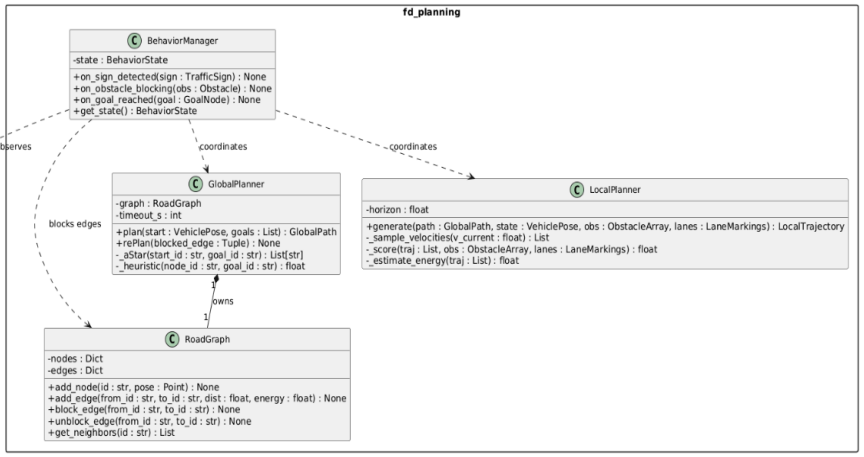
2. Packages

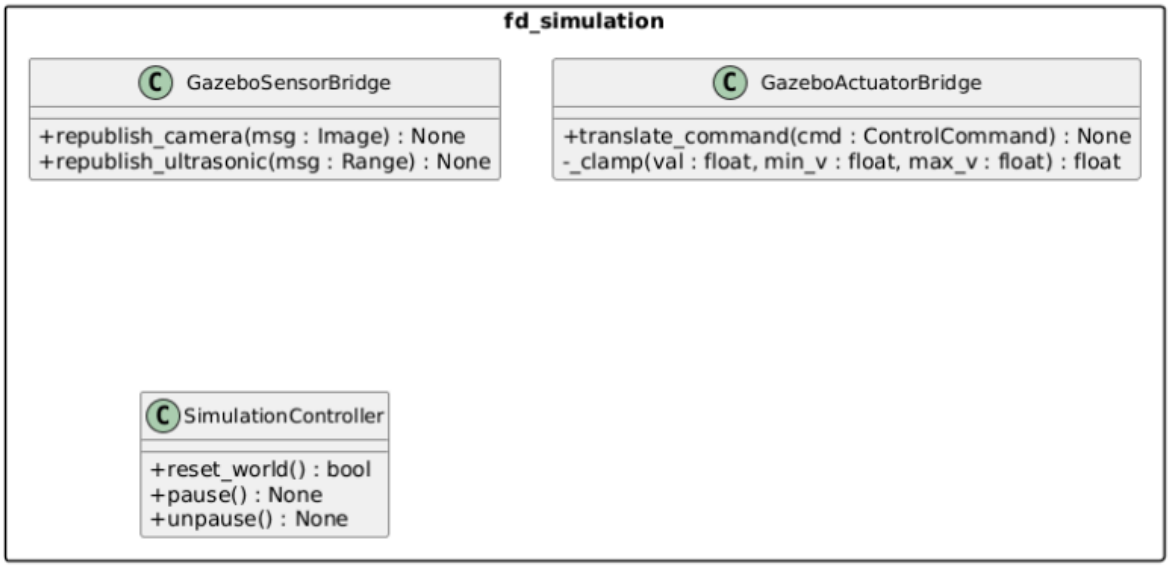
The F&D system is organized into six ROS2 packages and one shared message package. This structure follows the single-responsibility principle and mirrors the subsystem decomposition from the HLD.

Package	Responsibility
fd_msgs	Defines all custom ROS2 message, service, and action types shared across subsystems.
fd_perception	Vision processing, obstacle classification, lane detection, traffic sign recognition, localization.
fd_planning	Global path planning (A*), local trajectory generation, behavior management.
fd_control	Trajectory tracking (PID/MPC), emergency stop, actuator command publication.
fd_simulation	Sensor republishing, control command translation, simulation lifecycle management.
fd_launch	Launch files, parameter files, and environment configuration.


fd_control
ParkingController
+detect_bay(img : ndarray) : ParkingBay
+plan_maneuver(bay : ParkingBay, pose : VehiclePose) : LocalTrajectory
+verify_parked(pose : VehiclePose, bay : ParkingBay) : bool
EmergencyStop
-threshold : float
+check_and_trigger(obs : ObstacleArray, pose : VehiclePose, v : float) : bool
-_time_to_collision(obs : Obstacle, pose : VehiclePose, v : float) : float
+reset() : None
TrajectoryTracker
-K_p : float
-K_i : float
-K_d : float
+control_loop(traj : LocalTrajectory, pose : VehiclePose) : ControlCommand
-_compute_lateral_error(pose : VehiclePose, traj : LocalTrajectory) : float
-_compute_heading_error(pose : VehiclePose, traj : LocalTrajectory) : float
-_pid_steer(lat_err : float, head_err : float) : float
-_compute_throttle_brake(v_cur : float, v_tgt : float) : Tuple


fd_perception
VisionProcessor
-model_path : str
-conf_threshold : float
+run_pipeline(msg : Image) : None
+detect_lanes(img : ndarray) : LaneMarkings
«Singleton»
LocalizationEstimator
-pose : VehiclePose
+update(gt_pose : Pose, sigma : float) : VehiclePose
+get_pose() : VehiclePose
+reset(initial_pose : Pose) : None
uses
uses
uses
observes
ObstacleClassifier
+classify(img : ndarray) : Obstacle
DepthEstimator
+estimate(obs : Obstacle, img : ndarray) : float
-_monocular_depth(bbox : Rect, h : float) : float
TrafficSignRecognizer
+detect_signs(dets : List, img : ndarray) : List[TrafficSign]
-_validate_stop_sign(crop : ndarray) : float


fd_planning
BehaviorManager
-state : BehaviorState
+on_sign_detected(sign : TrafficSign) : None
+on_obstacle_blocking(obs : Obstacle) : None
+on_goal_reached(goal : GoalNode) : None
+get_state() : BehaviorState
coordinates
coordinates
blocks edges
GlobalPlanner
-graph : RoadGraph
-timeout_s : int
+plan(start : VehiclePose, goals : List) : GlobalPath
+rePlan(blocked_edge : Tuple) : None
-_aStar(start_id : str, goal_id : str) : List[str]
-_heuristic(node_id : str, goal_id : str) : float
LocalPlanner
-horizon : float
+generate(path : GlobalPath, state : VehiclePose, obs : ObstacleArray, lanes : LaneMarkings) : LocalTrajectory
-_sample_velocities(v_current : float) : List
-_score(traj : List, obs : ObstacleArray, lanes : LaneMarkings) : float
-_estimate_energy(traj : List) : float
owns
1
RoadGraph
-nodes : Dict
-edges : Dict
+add_node(id : str, pose : Point) : None
+add_edge(from_id : str, to_id : str, dist : float, energy : float) : None
+block_edge(from_id : str, to_id : str) : None
+unblock_edge(from_id : str, to_id : str) : None
+get_neighbors(id : str) : List


fd_simulation
GazeboSensorBridge
+republish_camera(msg : Image) : None
+republish_ultrasonic(msg : Range) : None
GazeboActuatorBridge
+translate_command(cmd : ControlCommand) : None
-_clamp(val : float, min_v : float, max_v : float) : float
SimulationController
+reset_world() : bool
+pause() : None
+unpause() : None

3. Class Interfaces

Each subsystem is further decomposed into classes. This section specifies the interface of every significant class: its responsibilities, design patterns applied, public methods, and its ROS2 topics.

3.1 Perception Subsystem (fd_perception)

3.1.1 VisionProcessor

Responsibilities: Subscribes to raw camera images, runs the detection model, and publishes detected object arrays and lane markings.

Method Signature	Pre-condition	Post-condition / Returns
__init__(model_path: str, conf_threshold: float = 0.5)	model_path points to a valid YOLO weights file.	Model loaded
_preprocess(img: np.ndarray) -> torch.Tensor	img is a valid H×W×3 array.	Returns a normalized tensor ready for inference.
_infer(tensor: torch.Tensor) -> List[Detection]	tensor shape matches model input.	Returns list of Detection(bbox, class_id, confidence) objects.
_postprocess(detections: List[Detection], img_shape) -> ObstacleArray	detections are non-None.	Returns ROS2 ObstacleArray message with 3D projected bounding boxes.
detect_lanes(img: np.ndarray) -> LaneMarkings	img is valid.	Returns LaneMarkings with left, right, center polylines; raises InsufficientDataError if no lanes are found.
run_pipeline(msg: Image) -> None	Node initialized.	Publishes the obstacles and llanes at >= 10 FPS.

3.1.2 ObstacleClassifier

Responsibilities: Receives raw detections and classifies them.

Method Signature	Pre-condition	Post-condition / Returns
classify(img: np.ndarray,) -> Obstacle	detection has a valid bbox.	Returns Obstacle with type with estimated distance to the object

3.1.3 TrafficSignRecognizer

Responsibilities: Identifies STOP and PARK signs, with position and confidence.

Method Signature	Pre-condition	Post-condition / Returns
detect_signs(detections: List[Detection], img: np.ndarray) -> List[TrafficSign]	detections is a non-empty list.	Returns list of TrafficSign; each has type, estimated pose, confidence. Publishes the sign data.
_validate_stop_sign(crop: np.ndarray) -> float	The cropped image is valid.	Returns confidence in [0,1] that crop is a red hexagon stop sign on yellow board at 0.5–1.0m elevation.

3.1.4 DepthEstimator

Responsibilities: Fuses ultrasonic readings and visual cues to estimate object distances.

Method Signature	Pre-condition	Post-condition / Returns
estimate(obstacle: Obstacle, cameraImage: np.ndarray) -> float	obstacle.bbox and image are valid .	Returns distance in metres with <= 20% error. Raises SensorFusionError if any of the sources are unavailable.
_monocular_depth(bbox: Rect, known_height_m: float) -> float	known_height_m > 0.	Returns depth estimate using apparent-size heuristic.

3.1.5 LocalizationEstimator

Responsibilities: Maintains vehicle pose estimate, publishes at >= 1 Hz with position error <= 0.50m.

Method Signature	Pre-condition	Post-condition / Returns
update(gt_pose:Pose, noise_sigma: float = 0.05) -> VehiclePose	gt_pose valid.	Updates internal state; returns noisy VehiclePose; publishes to /perception/vehicle_pose.
get_pose() -> VehiclePose	Node initialized.	Returns the latest pose.
reset(initial_pose: Pose) -> None	Called on system startup.	Resets internal state to initial_pose with zero velocity.

3.2 Planning Subsystem (fd_planning)

3.2.1 RoadGraph

Responsibilities: Represents the navigable road network as a weighted directed graph. Supports dynamic blocking of segments when obstacles are detected.

Method Signature	Pre-condition	Post-condition / Returns
__init__(world_file: str) -> None	world_file is a valid Gazebo .sdf file.	Graph loaded; all nodes and edges initialized with distances and energy weights.
add_node(node_id: str, pose: Point) -> None	node_id unique.	Node added to graph.
add_edge(from_id: str, to_id: str, distance: float, energy_weight: float, passable: bool = True) -> None	Both nodes exist.	Directed edge created; energy_weight reflects road curvature and gradient.
block_edge(from_id: str, to_id: str) -> None	Edge exists.	Sets edge passable=False; triggers GlobalPlanner replan event.
unblock_edge(from_id: str, to_id: str) -> None	Edge exists.	Restores edge passable=True.
get_neighbors(node_id: str) -> List[Tuple[str, float, float]]	node_id is in the graph.	Returns list of (neighbor_id, distance, energy_weight) for passable edges only.

3.2.2 GlobalPlanner

Responsibilities: Computes a full route visiting all goal nodes using A*. Must complete within 10 seconds. Publishes a GlobalPath; replans when RoadGraph signals a blockage.

Method Signature	Pre-condition	Post-condition / Returns
plan(start_pose: VehiclePose, goals: List[GoalNode]) -> GlobalPath	start_pose valid; goals non-empty; graph loaded.	Returns GlobalPath; raises PlanningTimeoutError if computation exceeds 10 s; raises NoValidPathError if no route exists.
_aStar(start_id: str, goal_id: str) -> List[str]	Both IDs in graph.	Returns ordered list of node IDs forming lowest-cost path; cost = distance + energy_weight .
_heuristic(node_id: str, goal_id: str) -> float	Both IDs in graph.	Returns Euclidean distance as admissible heuristic.
rePlan(blocked_edge: Tuple[str, str]) -> None	Called by RoadGraph observer callback.	Re-runs plan() from current vehicle position with updated graph; publishes new GlobalPath within 10 s.

3.2.3 LocalPlanner

Responsibilities: Generates short-term trajectories (3-second horizon), avoiding obstacles, keeping lanes, obeying signs, and minimizing energy cost.

Method Signature	Pre-condition	Post-condition / Returns
generate(global_path: GlobalPath, vehicle_state: VehiclePose, obstacles: ObstacleArray, lanes: LaneMarkings) -> LocalTrajectory	All inputs current (< 3000ms old).	Returns best-scored LocalTrajectory; updates every <= 3 s. Raises InsufficientDataError if inputs are stale.
_sample_velocities(v_current: float) -> List[Tuple[float, float]]	v_current in [0, v_max]; omega in valid range.	Returns N (v, omega) pairs within dynamic constraints.
_simulate_trajectory(v: float, omega: float, horizon: float = 3.0) -> List[Pose]	v, omega in valid ranges.	Simulates vehicle kinematics for horizon seconds; returns pose sequence.
_score(trajectory: List[Pose], obstacles: ObstacleArray, lanes: LaneMarkings, global_path: GlobalPath) -> float	All inputs valid.	Returns scalar cost.
_estimate_energy(trajectory: List[Pose]) -> float	trajectory has >= 2 poses.	Returns proxy energy cost based on total acceleration magnitude along trajectory.

3.2.4 BehaviorManager

Responsibilities: Coordinates high-level behavior: stop-sign compliance, parking maneuver initiation, global replan triggers, and task completion detection.

State	Entry Action	Transitions Out
DRIVING	Enable LocalPlanner normal mode.	STOP sign detected within 10m → STOPPING; Obstacle blocks path → REPLANNING; Parking bay detected → PARKING; All goals visited → TASK_COMPLETE
STOPPED_AT_SIGN	Start 3-second timer.	Timer elapsed → DRIVING
PARKING	Switch LocalPlanner to parking mode; allow reverse.	Parking complete (tolerance met) → END_DRIVING
REPLANNING	Block current edge in RoadGraph; request GlobalPlanner replan.	New GlobalPath received → DRIVING; No valid path → TASK_FAILED
TASK_COMPLETE / TASK_FAILED	Publish completion metrics; stop vehicle.	

Method Signature	Pre-condition	Post-condition / Returns
on_sign_detected(sign: TrafficSign) -> None	State = DRIVING.	Transitions to STOPPING if STOP sign; to PARKING if PARKING bay.
on_obstacle_blocking(obstacle: Obstacle) -> None	State = DRIVING.	Transitions to REPLANNING; blocks relevant edges in RoadGraph.
on_goal_reached(goal: GoalNode) -> None	Goal in remaining goals list.	Marks goal visited; if all visited, transitions to TASK_COMPLETE.
get_state() -> BehaviorState	Always.	Returns current BehaviorState.

3.3 Control Subsystem (fd_control)

3.3.1 TrajectoryTracker

Responsibilities: Subscribes to LocalTrajectory and VehiclePose; outputs ControlCommand to stay on trajectory.

Method Signature	Pre-condition	Post-condition / Returns
control_loop(trajectory: LocalTrajectory, pose: VehiclePose) -> ControlCommand	trajectory and pose are current.	Returns ControlCommand with steering, throttle, brake.
_compute_lateral_error(pose: VehiclePose, trajectory: LocalTrajectory) -> float	Trajectory non-empty.	Returns signed lateral distance from nearest trajectory point in metres.
_compute_heading_error(pose: VehiclePose, trajectory: LocalTrajectory) -> float	Trajectory non-empty.	Returns heading difference in radians.
_pid_steer(lat_err: float, head_err: float) -> float	Errors in valid ranges.	Return steering angle in radians using PID gains K_p, K_i, K_d (tuned offline).
_compute_throttle_brake(v_current: float, v_target: float) -> Tuple[float, float]	Speeds >= 0.	Returns (throttle, brake) in [0,1] each; uses proportional speed controller.

3.3.2 EmergencyStop

Responsibilities: Monitors collision proximity; publishes maximum brake command overriding all other control outputs when imminent collision is detected.

Method Signature	Pre-condition	Post-condition / Returns
check_and_trigger(obstacles: ObstacleArray, pose: VehiclePose, velocity: float) -> bool	Always callable.	Returns True and publishes EMSTOP ControlCommand if nearest obstacle < 0.30m else False.
reset() -> None	Called when the obstacle clears.	Clears emergency state; resumes normal control.

3.3.3 ParkingController

Responsibilities: Executes the parking maneuver. Identifies bay boundaries (0.15m blue lines bordered by white), computes entry trajectory, and verifies final pose within tolerance (<= 0.15m lateral, <= 20° heading error).

Method Signature	Pre-condition	Post-condition / Returns
detect_bay(img: np.ndarray) -> Optional[ParkingBay]	camera must be providing images	Returns ParkingBay(entry_pose, width, depth) if blue-bounded bay found; else None.
plan_maneuver(bay: ParkingBay, vehicle_pose: VehiclePose) -> LocalTrajectory	bay exists near vehicle within 5m.	Returns trajectory with reverse phases; sets control mode to PARK.
verify_parked(vehicle_pose: VehiclePose, bay: ParkingBay) -> bool	Maneuver executed.	Returns True if the car is mostly inside the target parking bay.

3.4 Simulation Interface (fd_simulation)

3.4.1 GazeboSensorBridge

Responsibilities: Republishes Gazebo camera and ultrasonic data in standard formats; applies calibration transforms.

Method Signature	Pre-condition	Post-condition / Returns
republish_camera(gazebo_msg: sensor_msgs/Image) -> None	Gazebo camera plugin running.	Publishes with corrected color space and timestamp.
republish_ultrasonic(gazebo_msg: sensor_msgs/Range) -> None	Ultrasonic plugin running.	Publishes the clips to valid sensor range [0.02m, 4.0m].

3.4.2 GazeboActuatorBridge

Responsibilities: Converts fd_control ControlCommand messages into Gazebo joint and effort commands that the vehicle model's drive plugin can use.

Method Signature	Pre-condition	Post-condition / Returns
translate_command(cmd: ControlCommand) -> None	cmd.mode is valid.	Publishes Gazebo-compatible AckermannDriveStamped to vehicle plugin topic. In EMSTOP mode, publishes maximum brake.
_clamp(value: float, min_v: float, max_v: float) -> float	Always.	Returns value clamped to [min_v, max_v]; used for steering angle and throttle safety limits.

3.4.3 SimulationController

Responsibilities: Offers a single interface for launch files and the BehaviorManager to control the simulation lifecycle (start, pause, reset, set speed factor).

Method Signature	Pre-condition	Post-condition / Returns
reset_world() -> bool	Gazebo running.	Calls /reset_world service; returns True on success. Resets vehicle pose to initial position.
pause() -> None	Simulation running.	Pauses physics; sensor data freezes.
unpause() -> None	Simulation paused.	Resumes physics.

3.5 Design Patterns Summary

The following table consolidates all design patterns applied in the low-level design for traceability:

Class	Rationale
VisionProcessor	Supports swapping YOLO variants; standardizes image pipeline stages.
ObstacleClassifier	Decouples obstacle type creation logic from caller; extensible for new obstacle types.
TrafficSignRecognizer	Decouples sign detection from reactive behavior; multiple subscribers possible.
DepthEstimator	Hides sensor fusion complexity; provides single clean distance API.
LocalizationEstimator	Guarantees one consistent pose source throughout the system.
RoadGraph	Shares immutable road geometry; uniform traversal of nodes and edges.
GlobalPlanner	Algorithm replaceable; plan() enforces consistent workflow.
LocalPlanner	Candidate trajectories as independent command objects enable clean scoring.
BehaviorManager	Explicit states prevent undefined transitions; decoupled event handling.
TrajectoryTracker	Fixed loop structure; PID or MPC interchangeable at config time.
EmergencyStop	Highest-priority safety check; cleanly overrides normal commands.
ParkingController	Specialized controller injected only during parking phase.
GazeboSensorBridge	Isolates perception from Gazebo-specific APIs; eases simulator swap.
GazeboActuatorBridge	Isolates control from Gazebo vehicle plugin specifics.
SimulationController	Single interface to multiple Gazebo lifecycle services.

4. Glossary

Term	Definition
Adapter Pattern	Structural design pattern that converts an interface of a class into another interface expected by clients.
A* Algorithm	Informed graph search algorithm using f(n) = g(n) + h(n), where g is cost so far and h is the admissible heuristic.
Bbox	Axis-aligned bounding box defined by its top-left corner (x, y) and dimensions (width, height), all as floats.
Behavior Manager	The orchestrating component that coordinates subsystem states based on environmental events.
Chain of Responsibility	Behavioral pattern where a request passes through a chain of handlers; the first handler able to process it does so.
Command Pattern	Behavioral pattern that encapsulates a request as an object, allowing parameterization, queuing, and independent execution.
DWA (Dynamic Window Approach)	Local planner algorithm that samples feasible (v, omega) pairs within dynamic constraints and selects lowest-cost trajectory.
EmergencyStop	Safety-critical component that overrides all control outputs to apply maximum braking when collision is imminent.
Facade Pattern	Structural pattern providing a simplified interface to a complex subsystem.
Factory Method	Creational pattern defining an interface for object creation, allowing subclasses to decide which class to instantiate.
Flyweight Pattern	Structural pattern minimizing memory use by sharing common parts of state among many objects.
FR / NFR	Functional / Non-Functional Requirement as defined in the Analysis Report.
GlobalPlanner	Component computing full route through the road network visiting all goal nodes.
GoalNode	A target location on the map the vehicle must approach within 3 metres.
LocalTrajectory	Time-parameterized path for the next 3 seconds specifying vehicle position, heading, and velocity at each step.
MPC (Model Predictive Control)	Advanced control technique predicting future system states over a horizon and optimizing control inputs accordingly.

Term	Definition
Observer Pattern	Behavioral pattern where an object maintains a list of dependents and notifies them of state changes.
OBB (Oriented Bounding Box)	A bounding box with arbitrary rotation, used to represent obstacle geometry in world coordinates.
PID Control	Feedback controller computing corrections based on Proportional, Integral, and Derivative terms of the error signal.
QoS (Quality of Service)	ROS2 communication reliability policy; Reliable guarantees delivery; BestEffort sacrifices delivery for lower latency.
RoadGraph	Directed weighted graph representation of the navigable road network.
Singleton Pattern	Creational pattern ensuring a class has only one instance and providing a global access point to it.
State Machine Pattern	Behavioral pattern modeling an object's lifecycle as a set of explicit states with guarded transitions.
Strategy Pattern	Behavioral pattern defining a family of algorithms, encapsulating each and making them interchangeable.
TF2	ROS2 coordinate transform library managing spatial relationships between frames (e.g., camera_frame to base_link).
Template Method Pattern	Behavioral pattern defining the skeleton of an algorithm in a base class, deferring some steps to subclasses.
YOLO (You Only Look Once)	Real-time single-pass object detection neural network; YOLOv8 variant used in this project.

5. References

1. N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst., Sendai, Japan, 2004, pp. 2149–2154.
2. S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," Science Robotics, vol. 7, no. 66, May 2022.
3. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 779–788.
4. A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in Neural Information Processing Systems 32, Curran Associates, Inc., 2019, pp. 8024–8035.
5. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics. Cambridge, MA, USA: MIT Press, 2005.
6. X. Chen, H. Ma, J. Wan, B. Li, and T. Xia, "Multi-view 3D object detection network for autonomous driving," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2017.
7. E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
8. IEEE Std 1016-2009, IEEE Standard for Information Technology—Systems Design—Software Design Descriptions. IEEE, 2009.
9. Object Management Group, "Unified Modeling Language Specification Version 2.5.1," OMG Document formal/2017-12-05, 2017.
10. D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," IEEE Robotics & Automation Magazine, vol. 4, no. 1, pp. 23–33, Mar. 1997.
11. European Commission, "Ethics guidelines for trustworthy AI," 2020.
12. ROS2 Documentation, "ROS 2 Design," [Online]. Available: https://design.ros2.org/
13. OpenCV Documentation, "OpenCV Documentation," [Online]. Available: https://docs.opencv.org/
14. Ultralytics, "YOLOv8 Documentation," [Online]. Available: https://docs.ultralytics.com/